

```
11. #include <stdio.h>
```

```
#include <math.h>
```

```
int main()
```

```
{
```

```
    long double keys = 1.0;
```

```
    int i;
```

```
    for(i = 1; i <= 25; i++)
```

```
        keys *= i;
```

```
    printf("Number of possible keys = %.0Lf\n", keys);
```

```
    printf("Approximately 2^%.2Lf\n", log2l(keys));
```

```
    return 0;
```

```
}
```

```
Number of possible Playfair keys = 25! = 0
Approximately 2^0.00 keys
```

```
Process returned 0 (0x0)    execution time : 1.225 s
Press any key to continue.
```

```
12. #include <stdio.h>
```

```
#include <string.h>
```

```
#define MOD 26
```

```
int mod(int x)
```

```
{
```

```
    x %= MOD;
```

```
    if(x < 0)
```

```
        x += MOD;
```

```
    return x;
```

```
}
```

```
int inverseMod(int a)
```

```
{
```

```
    int i;
```

```
    for(i = 1; i < MOD; i++)
```

```
        if(mod(a * i) == 1)
```

```
            return i;
```

```
    return -1;
```

```
}
```

```

void encrypt(char *text, int key[2][2], char *cipher)
{
    int i, len = strlen(text);
    int p1, p2;

    for(i = 0; i < len; i += 2)
    {
        p1 = text[i] - 'a';
        p2 = text[i + 1] - 'a';

        cipher[i] = mod(key[0][0] * p1 +
                        key[0][1] * p2) + 'a';

        cipher[i + 1] = mod(key[1][0] * p1 +
                           key[1][1] * p2) + 'a';
    }

    cipher[len] = '\0';
}

```

```

void decrypt(char *cipher, int key[2][2], char *plain)
{
    int det, invDet, i, len = strlen(cipher);
    int inv[2][2];
    int c1, c2;

    det = mod(key[0][0] * key[1][1] -
              key[0][1] * key[1][0]);

    invDet = inverseMod(det);

    if(invDet == -1)
    {
        printf("Invalid key matrix\n");
        return;
    }

    inv[0][0] = mod(invDet * key[1][1]);
    inv[0][1] = mod(-invDet * key[0][1]);
    inv[1][0] = mod(-invDet * key[1][0]);
    inv[1][1] = mod(invDet * key[0][0]);

    for(i = 0; i < len; i += 2)
    {
        c1 = cipher[i] - 'a';

```

```

    c2 = cipher[i + 1] - 'a';

    plain[i] = mod(inv[0][0] * c1 +
                  inv[0][1] * c2) + 'a';

    plain[i + 1] = mod(inv[1][0] * c1 +
                      inv[1][1] * c2) + 'a';
}

plain[len] = '\0';
}

int main()
{
    int key[2][2] = {
        {9, 4},
        {5, 7}
    };

    char plaintext[] =
        "meetmeattheusualplaceattenratherthaneightoclock";

    char padded[200];
    char ciphertext[200];
    char decrypted[200];

    int len = strlen(plaintext);

    strcpy(padded, plaintext);

    if(len % 2 != 0)
    {
        padded[len] = 'x';
        padded[len + 1] = '\0';
    }

    printf("Plaintext : %s\n", padded);

    encrypt(padded, key, ciphertext);

    printf("Ciphertext : %s\n", ciphertext);

    decrypt(ciphertext, key, decrypted);

    printf("Decrypted : %s\n", decrypted);

```

```
return 0;
}
```

```
Plaintext : meetmeattheusualplaceattenratherthaneightoclockx
Ciphertext : ukixukydromeiwszxwiokunukhxxhroajroanqyebtlkjegad
Decrypted : meetmeattheusualplaceattenratherthaneightoclockx
```

```
Process returned 0 (0x0) execution time : 0.770 s
Press any key to continue.
```

13. #include <stdio.h>

#define MOD 26

```
int mod(int x)
{
    x %= MOD;
    if(x < 0)
        x += MOD;
    return x;
}
```

```
int inverseMod(int a)
{
    int i;

    for(i = 1; i < MOD; i++)
        if(mod(a * i) == 1)
            return i;

    return -1;
}
```

```
int main()
{
    int P[2][2] = {
        {7, 0},
        {8, 19}
    };

    int C[2][2] = {
        {19, 4},
        {2, 5}
    };

    int Pinv[2][2];
```

```

int K[2][2];

int det, invDet;

det = mod(P[0][0] * P[1][1] -
          P[0][1] * P[1][0]);

invDet = inverseMod(det);

if(invDet == -1)
{
    printf("Matrix is not invertible\n");
    return 0;
}

Pinv[0][0] = mod(invDet * P[1][1]);
Pinv[0][1] = mod(-invDet * P[0][1]);
Pinv[1][0] = mod(-invDet * P[1][0]);
Pinv[1][1] = mod(invDet * P[0][0]);

K[0][0] = mod(C[0][0] * Pinv[0][0] +
              C[0][1] * Pinv[1][0]);

K[0][1] = mod(C[0][0] * Pinv[0][1] +
              C[0][1] * Pinv[1][1]);

K[1][0] = mod(C[1][0] * Pinv[0][0] +
              C[1][1] * Pinv[1][0]);

K[1][1] = mod(C[1][0] * Pinv[0][1] +
              C[1][1] * Pinv[1][1]);

printf("Recovered Key Matrix:\n");

printf("%d %d\n", K[0][0], K[0][1]);
printf("%d %d\n", K[1][0], K[1][1]);

return 0;
}

```

```

D:\exp 3.exe
Process returned 15001 (0x3A99)    execution time : 1.019 s
Press any key to continue.

```

```

14. #include <stdio.h>
#include <string.h>
#include <ctype.h>

int main()
{
    char plaintext[] = "send more money";

    int key[] = {
        9, 0, 1, 7, 23, 15, 21,
        14, 11, 11, 2, 8, 9
    };

    char ciphertext[100];

    int i;
    int keyIndex = 0;

    for(i = 0; plaintext[i] != '\0'; i++)
    {
        if(isalpha(plaintext[i]))
        {
            int p = tolower(plaintext[i]) - 'a';
            int c = (p + key[keyIndex]) % 26;

            ciphertext[i] = c + 'a';

            keyIndex++;
        }
        else
        {
            ciphertext[i] = plaintext[i];
        }
    }

    ciphertext[i] = '\0';

    printf("Plaintext : %s\n", plaintext);
    printf("Ciphertext : %s\n", ciphertext);

    return 0;
}

```

```
"D:\exp 4.exe"
Plaintext : send more money
Ciphertext : beok jdms xzpmh

Process returned 0 (0x0)   execution time : 0.704 s
Press any key to continue.
|
```

```
15. #include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define MAX 1000
```

```
double freq[26] = {
    12.70, 9.06, 8.17, 7.51, 6.97, 6.75,
    6.33, 6.09, 5.99, 4.25, 4.03, 2.76,
    2.41, 2.36, 2.23, 1.97, 1.93, 1.91,
    1.29, 0.93, 0.98, 0.77, 0.15, 0.15,
    0.10, 0.07
};
```

```
void decrypt(char *cipher, int shift, char *plain)
```

```
{
    int i;

    for(i = 0; cipher[i] != '\0'; i++)
    {
        if(isalpha(cipher[i]))
        {
            char ch = tolower(cipher[i]);

            plain[i] =
                ((ch - 'a' - shift + 26) % 26) + 'a';
        }
        else
        {
            plain[i] = cipher[i];
        }
    }
}
```

```

    plain[i] = '\0';
}

double score(char *text)
{
    int count[26] = {0};
    int total = 0;
    int i;
    double result = 0;

    for(i = 0; text[i] != '\0'; i++)
    {
        if(isalpha(text[i]))
        {
            count[tolower(text[i]) - 'a']++;
            total++;
        }
    }

    if(total == 0)
        return 0;

    for(i = 0; i < 26; i++)
    {
        double observed =
            (double)count[i] / total * 100;

        double difference =
            observed - freq[i];

        if(difference < 0)
            difference = -difference;

        result += difference;
    }

    return result;
}

int main()
{
    char ciphertext[MAX];
    char plaintext[MAX];

    int shift;

```



```

printf("Enter ciphertext: ");
fgets(ciphertext, MAX, stdin);

for(shift = 0; shift < 26; shift++)
{
    decrypt(ciphertext, shift, plaintext);

    printf("Shift %2d : %.2f : %s",
        shift,
        score(plaintext),
        plaintext);
}

return 0;
}

```

```

Enter ciphertext: jdejfejffrf'ksjd
Shift 0 : 132.44 : jdejfejffrf'ksjd
Shift 1 : 124.40 : icdiedieeqe'jric
Shift 2 : 116.42 : hbchdchddpd'iqhb
Shift 3 : 104.76 : gabgcbgccoc'hpga
Shift 4 : 118.86 : fzaafbafbbnb'gofz
Shift 5 : 135.31 : eyzeazeaama'fney
Shift 6 : 158.53 : dxydzydzzlz'emdx
Shift 7 : 153.81 : cwxcyxcyyky'dlcw
Shift 8 : 147.71 : bvwbxwbxxjx'ckbv
Shift 9 : 134.85 : auvawvawwiw'bjau
Shift 10 : 154.87 : ztuzvuzvvhv'aizt
Shift 11 : 166.28 : ystyutyuugu'zhys
Shift 12 : 163.11 : xrsxtsxttft'ygxr
Shift 13 : 160.33 : wqrwsrwsses'xfwq
Shift 14 : 157.73 : vpqvrqvrdr'wepv
Shift 15 : 155.43 : uopuquqqcq'vduo
Shift 16 : 154.25 : tnotpotppbp'uctn
Shift 17 : 152.75 : smnsonsooao'tbsm
Shift 18 : 162.93 : rlmrnmrnnzn'sarl
Shift 19 : 171.44 : qklqmlqmmym'rzqk
Shift 20 : 167.48 : pjklplkpllxl'qypj
Shift 21 : 160.32 : oijokjokkww'pxoi
Shift 22 : 154.18 : nhinjinjjvj'ownh
Shift 23 : 148.00 : mghmihmiiui'nvmg
Shift 24 : 145.36 : lfghlghhhth'mulf
Shift 25 : 139.74 : kefkfgkkgsg'ltke

```

```

Process returned 0 (0x0)   execution time : 0.729 s

```

```

16. #include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 1000

double freq[26] = {
    12.70, 9.06, 8.17, 7.51, 6.97, 6.75,
    6.33, 6.09, 5.99, 4.25, 4.03, 2.76,
    2.41, 2.36, 2.23, 1.97, 1.93, 1.91,
    1.29, 0.93, 0.98, 0.77, 0.15, 0.15,
    0.10, 0.07
};

void countFrequency(char *text, int count[])
{
    int i;

    for(i = 0; i < 26; i++)
        count[i] = 0;

    for(i = 0; text[i] != '\0'; i++)
    {
        if(isalpha(text[i]))
            count[tolower(text[i]) - 'a']++;
    }
}

int main()
{
    char cipher[MAX];
    int count[26];
    int i, j, temp;
    char letters[26];

    printf("Enter ciphertext:\n");
    fgets(cipher, MAX, stdin);

    countFrequency(cipher, count);

    for(i = 0; i < 26; i++)
        letters[i] = 'a' + i;

    for(i = 0; i < 25; i++)
    {
        for(j = i + 1; j < 26; j++)

```

```

    {
        if(count[i] < count[j])
        {
            temp = count[i];
            count[i] = count[j];
            count[j] = temp;

            temp = letters[i];
            letters[i] = letters[j];
            letters[j] = temp;
        }
    }
}

printf("\nFrequency order:\n");

for(i = 0; i < 26; i++)
{
    printf("%c : %d\n", letters[i], count[i]);
}

printf("\nMost frequent letter is likely mapped to E.\n");

return 0;
}

```

```

Enter ciphertext:
fejfkefied

```

```

Frequency order:

```

```

e : 3
f : 3
d : 1
i : 1
j : 1
k : 1
g : 0
h : 0
a : 0
b : 0
c : 0
l : 0
m : 0
n : 0
o : 0
p : 0
q : 0
r : 0
s : 0
t : 0
u : 0
v : 0
w : 0
x : 0
y : 0
z : 0

```

17. #include <stdio.h>

```
int shiftSchedule[16] =  
{  
    1, 1, 2, 2, 2, 2, 2, 2,  
    1, 2, 2, 2, 2, 2, 2, 1  
};
```

```
void leftShift(char *key, int n, int shift)  
{  
    char temp[28];  
    int i;  
  
    for(i = 0; i < 28; i++)  
        temp[i] = key[i];  
  
    for(i = 0; i < 28; i++)  
        key[i] = temp[(i + shift) % 28];  
}
```

```
int main()  
{  
    char C[28], D[28];  
    char tempC[28], tempD[28];  
  
    int i, j;  
  
    printf("Enter 56-bit DES key:\n");  
  
    char key[57];  
    scanf("%56s", key);  
  
    for(i = 0; i < 28; i++)  
        C[i] = key[i];  
  
    for(i = 0; i < 28; i++)  
        D[i] = key[i + 28];  
  
    for(i = 0; i < 16; i++)  
    {  
        leftShift(C, 28, shiftSchedule[i]);  
        leftShift(D, 28, shiftSchedule[i]);  
  
        for(j = 0; j < 28; j++)
```

```

    {
        tempC[j] = C[j];
        tempD[j] = D[j];
    }

    printf("\nDecryption Round %d\n", 16 - i);

    printf("C = ");
    for(j = 0; j < 28; j++)
        printf("%c", tempC[j]);

    printf("\nD = ");
    for(j = 0; j < 28; j++)
        printf("%c", tempD[j]);

    printf("\n");
}

return 0;
}

```

```

Enter 56-bit DES key:
jifjrofj

Decryption Round 16
C = ifjrofj26|~j
D = ;°P6|~"?J~

Decryption Round 15
C = fjrofj26|~ji
D = ;°P6|~"?J~

Decryption Round 14
C = rofj26|~jifj
D = ;°P6|~"?J~

Decryption Round 13
C = fj26|~jifjro
D = ;°P6|~"?J~ P

Decryption Round 12
C = 26|~jifjrofj
D = °P6|~"?J~ ;

Decryption Round 11
C = 6|~jifjrofj2
D = P6|~"?J~ ;°

Decryption Round 10
C = ~jifjrofj26|
D = P6|~"?J~ ;°

```

18. #include <stdio.h>

```
int main()
{
    char key[57];
    char C[29], D[29];
    char subkey[49];

    int i, j;

    printf("Enter 56-bit key:\n");
    scanf("%56s", key);

    for(i = 0; i < 28; i++)
        C[i] = key[i];

    C[28] = '\0';

    for(i = 0; i < 28; i++)
        D[i] = key[i + 28];

    D[28] = '\0';

    printf("\nC and D halves:\n");

    printf("C = %s\n", C);
    printf("D = %s\n", D);

    for(i = 0; i < 48; i++)
    {
        if(i < 24)
            subkey[i] = C[i % 28];
        else
            subkey[i] = D[(i - 24) % 28];
    }

    subkey[48] = '\0';

    printf("\n48-bit Subkey:\n");

    for(j = 0; j < 48; j++)
        printf("%c", subkey[j]);

    printf("\n");

    return 0;
```

```

}
Enter 56-bit key:
bjlkljjggf

C and D halves:
C = bjlkljjggf
D =

48-bit Subkey:
≈≡ú>9jjggfa÷?%h

Process returned 0 (0x0)   execution time : 4.210 s
Press any key to continue.
|

```

19. #include <stdio.h>

#include <stdint.h>

```

uint64_t encryptBlock(uint64_t data, uint64_t k1,
                      uint64_t k2, uint64_t k3)

```

```

{
    data ^= k1;
    data = ~data;
    data ^= k2;
    data = ~data;
    data ^= k3;

```

```

    return data;

```

```

}

```

```

uint64_t cbcEncrypt(uint64_t plaintext,
                    uint64_t previous,
                    uint64_t k1,
                    uint64_t k2,
                    uint64_t k3)

```

```

{
    uint64_t x;

```

```

    x = plaintext ^ previous;

```

```

    return encryptBlock(x, k1, k2, k3);

```

```

}

```

```

int main()

```

```

{
    uint64_t plaintext[3];
    uint64_t ciphertext[3];

```

```

uint64_t k1 = 0x123456789ABCDEF0;
uint64_t k2 = 0x0FEDCBA987654321;
uint64_t k3 = 0x1122334455667788;

uint64_t iv = 0xAABBCCDDEEFF0011;

int i;

printf("Enter 3 plaintext blocks in hexadecimal:\n");

for(i = 0; i < 3; i++)
    scanf("%llx", &plaintext[i]);

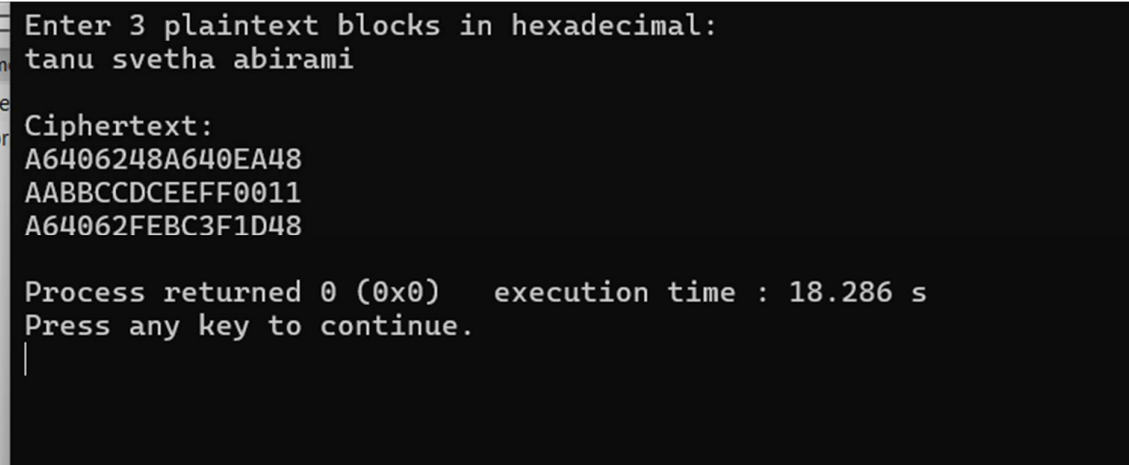
for(i = 0; i < 3; i++)
{
    if(i == 0)
        ciphertext[i] =
            cbcEncrypt(plaintext[i], iv,
                k1, k2, k3);
    else
        ciphertext[i] =
            cbcEncrypt(plaintext[i],
                ciphertext[i - 1],
                k1, k2, k3);
}

printf("\nCiphertext:\n");

for(i = 0; i < 3; i++)
    printf("%016llx\n", ciphertext[i]);

return 0;
}

```



```

Enter 3 plaintext blocks in hexadecimal:
tanu svetha abirami

Ciphertext:
A6406248A640EA48
AABBCCDCEEFF0011
A64062FEB3F1D48

Process returned 0 (0x0)   execution time : 18.286 s
Press any key to continue.
|

```


20. #include <stdio.h>

```
int main()
{
    int errorBlock;

    printf("Enter block containing ciphertext error: ");
    scanf("%d", &errorBlock);

    printf("\nECB Mode:\n");

    printf("Only plaintext block P%d is affected.\n",
        errorBlock);

    printf("\nCBC Mode:\n");

    printf("P%d is completely corrupted.\n",
        errorBlock);

    printf("The next plaintext block P%d is affected by\n",
        errorBlock + 1);

    printf("the corresponding bit error.\n");

    printf("\nBlocks after P%d are not affected.\n",
        errorBlock + 1);

    printf("\nIf there is a bit error in plaintext P1:\n");

    printf("The error affects ciphertext C1.\n");

    printf("During CBC decryption, P1 is affected.\n");

    printf("The changed bit also affects P2.\n");

    printf("Blocks P3 and beyond remain unaffected.\n");

    return 0;
}
```

```
s  D:\cryptography\lab\exp 20.  X  +  v

Enter block containing ciphertext error: kihhkkpl

ECB Mode:
Only plaintext block P0 is affected.

CBC Mode:
P0 is completely corrupted.
The next plaintext block P1 is affected by
the corresponding bit error.

Blocks after P1 are not affected.

If there is a bit error in plaintext P1:
The error affects ciphertext C1.
During CBC decryption, P1 is affected.
The changed bit also affects P2.
Blocks P3 and beyond remain unaffected.

Process returned 0 (0x0)    execution time : 7.771 s
Press any key to continue.
```